Universidad Metropolitana de Caracas
Facultad de Ingeniería
Sistemas Distribuidos

Sistema de Detección y Evasión en Estrella ("Aegis-Star 8-Sector")

1. Descripción General

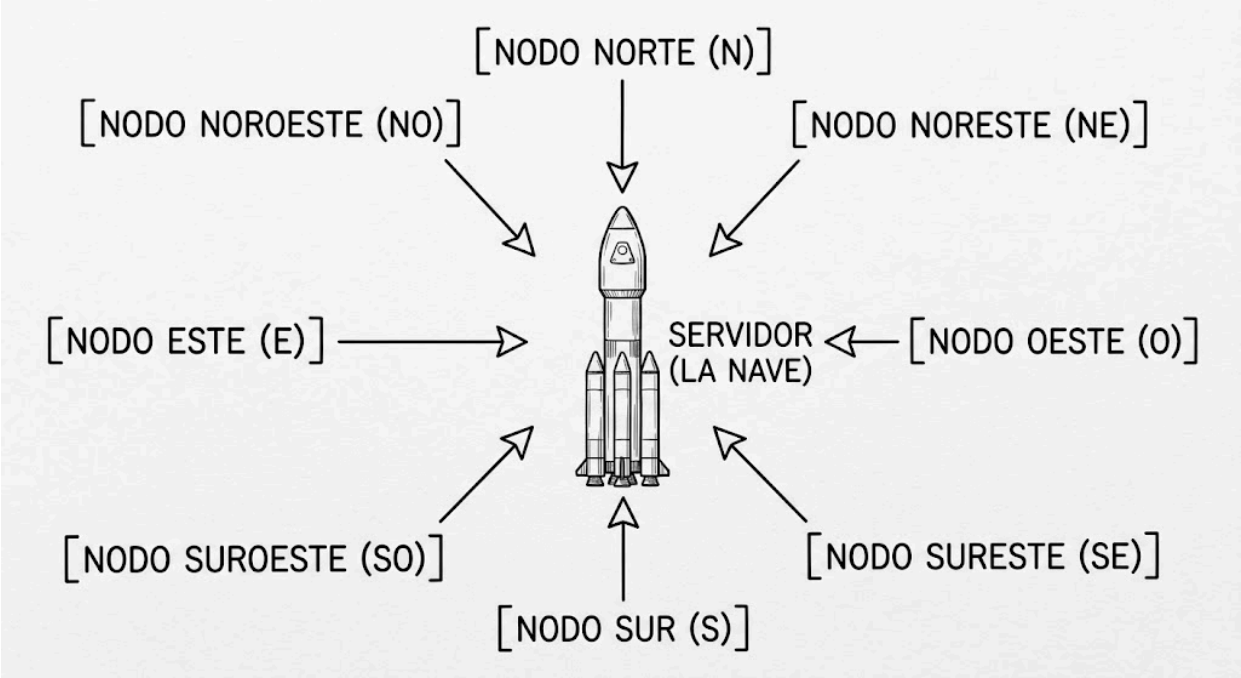
En este proyecto, los estudiantes desarrollarán un sistema distribuido tolerante a fallos con topología en estrella. El sistema simula el Centro de Mando de una nave espacial (Servidor Central) rodeado por una red de 8 sensores de radar periféricos (Nodos Sensores).

Los sensores escanean el espacio exterior en búsqueda de amenazas (asteroides, basura espacial) y reportan alertas en tiempo real al Servidor Central a través de mensajes usando sockets. El Servidor Central debe procesar los eventos en paralelo, eliminar duplicados, priorizar amenazas y ejecutar maniobras de evasión o activación de escudos.

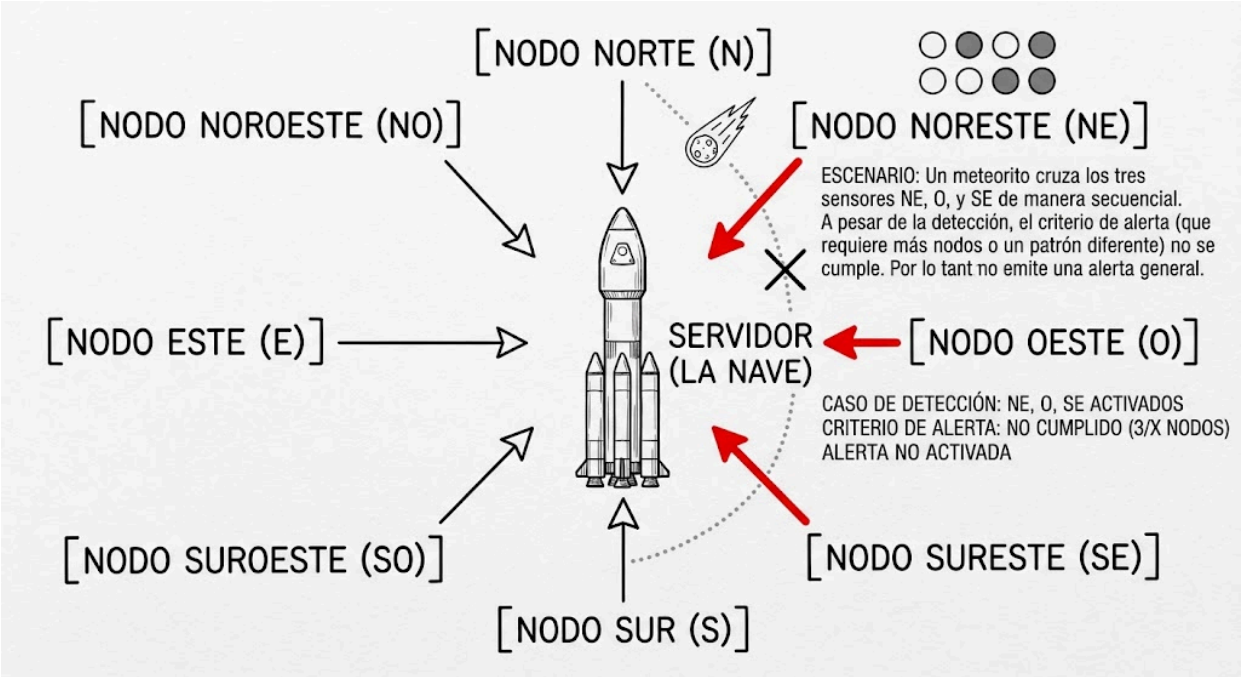
2. Objetivos del Proyecto

- **General:** Diseñar e implementar una arquitectura en estrella de 8 nodos sobre *sockets* para la gestión concurrente de eventos en tiempo real.
- **Específicos:**
    1. Implementar un protocolo de aplicación sobre sockets.
    2. Aplicar el algoritmo de número pseudoaleatorio (**PRNG con Semilla Compartida**) para la generación distribuida y determinista del entorno.
    3. Resolver condiciones de carrera, duplicación de eventos concurrentes y priorización de mensajes en el Servidor Central.
    4. Garantizar tolerancia a fallas mediante detección de desconexiones (*heartbeats*) y reconfiguración dinámica de cobertura.
    5. Suponer que los sensores pueden ser controlados por aliens y pueden fallar

3. Arquitectura del Sistema y Sectores (8 Nodos)

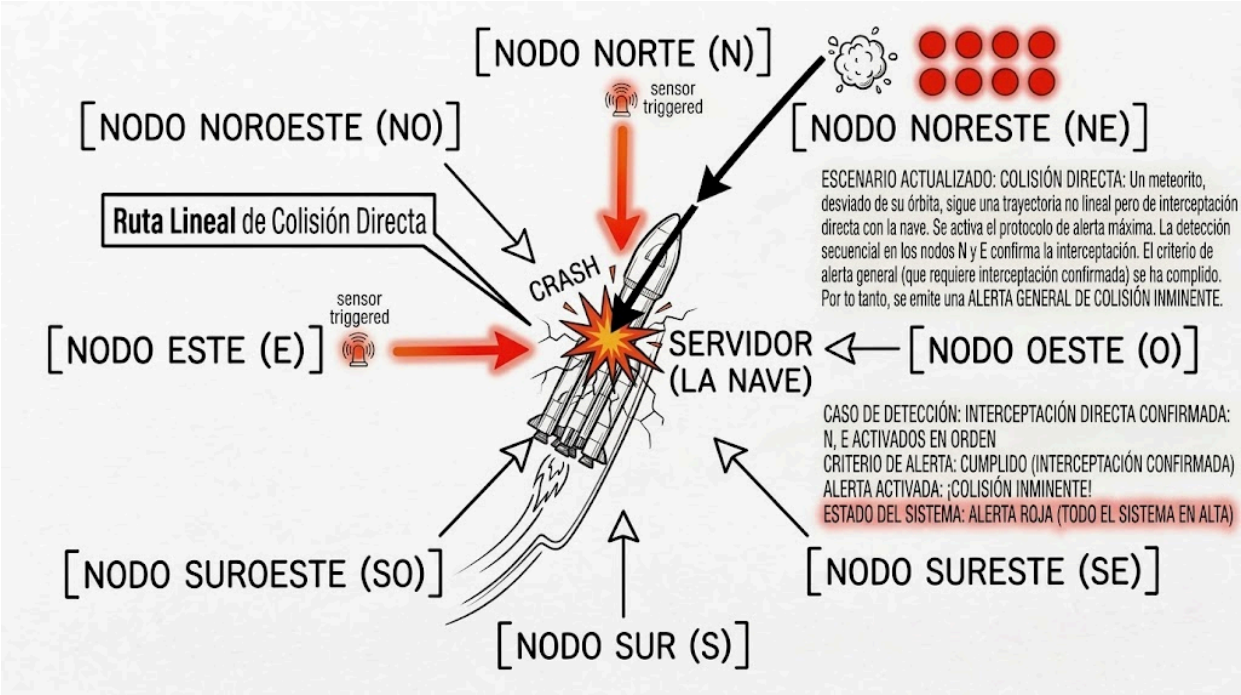


Por ejemplo, un meteorito cruza el espacio desplazándose de manera continua y paralela a la trayectoria de la nave. A medida que avanza, el objeto es detectado secuencialmente por tres de los sensores del sistema: el Nodo Noreste (NE), el Nodo Oeste (O) y el Nodo Sureste (SE).



Aunque los sensores registran la presencia del meteorito, el algoritmo de seguridad del servidor central no activa la alerta general. Esto se debe a que la trayectoria paralela no representa un

riesgo directo de colisión inminente o requiere la validación de un número mayor de nodos (o un patrón de interceptación específico) para disparar la alarma.



Un meteorito avanza en una trayectoria estrictamente lineal y directa hacia el servidor central (la nave). En su trayectoria recta de colisión, el objeto cruza secuencialmente los vectores de los sensores alineados en su ruta, como el Nodo Norte (N) y el Nodo Este (E), dirigiéndose sin desviación al punto central.

Matriz de Amenazas

La Matriz de Amenazas es un modelo binario de 3×3 que monitorea la rejilla espacial alrededor del servidor central de la nave espacial, donde cada celda equivale a un nodo de detección direccional ordenado desde el Noroeste hasta el Sureste. Dentro de esta estructura, un valor 0 indica ausencia de riesgo y un valor 1 señala la detección activa de un meteorito. La evaluación del nivel de peligro depende exclusivamente de la configuración de estos valores en los vectores de entrada para determinar si la ruta representa un impacto directo o un simple tránsito tangencial.

Cuando la matriz registra un único 1 aislado en cualquier nodo de entrada, el sistema interpreta que el objeto sigue una trayectoria lineal convergente directamente hacia el centro donde se ubica la nave, activando de inmediato la alerta roja por colisión inminente. Por el contrario, si se detectan dos o más valores 1 consecutivos y adyacentes en la secuencia de paso, la regla establece que no existe amenaza real, ya que este patrón continuo demuestra que el meteorito se desplaza de manera paralela o tangente cruzando los sensores de lado sin rumbo de interceptación.

Casos de ejemplo

ESTADO 1: SIN AMENAZAS

```
[ NO  N  NE ]    [ 0  0  0 ]
[  O  x  E  ]    [ 0  0  0 ]
[ SO  S  SE ]    [ 0  0  0 ]
```

ESTADO 2: OBJETO DESDE N AMENAZA

```
[ NO  N  NE ]    [ 0  1  0 ]
[  O  x  E  ]    [ 0  0  0 ]
[ SO  S  SE ]    [ 0  0  0 ]
```

ESTADO 3: SIN AMENZA OBJETO EN LA COLUMNA ESTE (NE, E, SE)

```
[ NO  N  NE ]    [ 0  0  1 ]
[  O  x  E  ]    [ 0  0  1 ]
[ SO  S  SE ]    [ 0  0  0 ]
```

ESTADO 4: OBJETO DESDE O

```
[ NO  N  NE ]    [ 0  0  0 ]
[  O  x  E  ]    [ 1  0  0 ]
[ SO  S  SE ]    [ 0  0  0 ]
```

ESTADO 5: AMENAZA OBJETO DESDE NO

```
[ NO  N  NE ]    [ 1  0  0 ]
[  O  x  E  ]    [ 0  0  0 ]
[ SO  S  SE ]    [ 0  0  0 ]
```

Responsabilidades de los Componentes:

- **Servidor Central (La Nave):**
    - Escucha conexiones entrantes de los 8 nodos sensores.
    - Implementa la matriz de amenazas
- **Nodos Sensores (Escáneres de Sector):**
    - Genera objetos de manera aleatoria y los reporta a la Nave.

6. Entregables

1. **Código Fuente:** Código fuente con La Nave y los Sensores
2. **Prueba de Evaluación (Demo):** Video acompañando el código mostrando su ejecución.
3. **Fecha de entrega**: en Moodle

7. Rúbrica de Evaluación

Aquí tienes la rúbrica presentada en formato de tabla para facilitar su consulta y aplicación:

Criterio / Ponderación	Excelente	Aceptable	Ineficiente	Falta de Todo
Ejecución y Video Demostrativo (25 puntos)	25 pts El programa compila y corre perfectamente con sockets. El video demostrativo es claro, de duración suficiente y evidencia en tiempo real el funcionamiento, la interacción entre procesos y la toma de decisiones.	17 pts El programa ejecuta con ajustes menores. El video es funcional pero apresurado, omitiendo detalles clave de la ejecución o de la explicación.	10 pts El programa presenta fallos de ejecución intermitentes, baja eficiencia en el entorno sockets o el video es muy deficiente para mostrar el flujo.	0 pts El programa no compila o presenta un fallo crítico (runtime error). No se entrega el video demostrativo.

Código: Nave y Sensores (25 puntos)	25 pts Difiere con claridad el rol del proceso servidor (la nave) y los procesos nodos (los sensores). El código de ambos está limpio, bien modularizado, estructurado y documentado.	17 pts Distingue entre la nave y los sensores, pero presenta duplicación de código, falta de modularidad o una estructura de procesos mejorable.	10 pts La separación entre el servidor y los sensores es confusa, el código carece de estructura adecuada y apenas incluye documentación.	0 pts No hay separación de responsabilidades entre la nave y los sensores. El código está desorganizado y no documentado.
Paso de Mensajes y Matriz de Amenazas (20 puntos)	20 pts Utiliza correctamente el paso de mensajes para recolectar detecciones. La nave construye dinámicamente la matriz de 3×3 sin pérdida de datos, condiciones de carrera ni deadlocks.	14 pts El paso de mensajes funciona en la mayoría de los casos y la matriz se construye, pero hay ineficiencias en las comunicaciones o leves desfases de sincronización.	10 pts Existe sobrecarga excesiva en la comunicación, desfases severos en la recepción de mensajes o la matriz se construye de forma ineficiente.	0 pts Fallos graves en el envío/recepción de datos entre sensores y la nave. La matriz no se construye o el programa cae en bloqueos mutuos (deadlocks).
Lógica de Clasificación y Decisiones (15 puntos)	15 pts Aplica la regla con precisión: un único 1 aislado activa Alerta Roja (colisión inminente) y dos o más 1 consecutivos se registran como tránsito seguro. Decisiones correctas en el 100% de los casos.	10 pts Toma decisiones correctas en escenarios simples, pero falla en la evaluación de casos límite (ej. consecutividad en bordes de la matriz o ráfagas rápidas).	10 pts La lógica de evaluación es ineficiente o inconsistente, generando un margen alto de falsas alarmas o evaluaciones erróneas.	0 pts La lógica para interpretar la matriz de amenazas no está implementada o es totalmente errónea, ignorando las colisiones directas.

Generación Aleatoria de Objetos (15 puntos)	15 pts Implementa de forma eficiente y correcta la generación aleatoria de meteoritos (trayectorias, origen y frecuencia) para simular un entorno dinámico e impredecible.	10 pts La generación aleatoria funciona, pero muestra patrones repetitivos, falta de variabilidad en las trayectorias o parámetros muy limitados.	10 pts La generación aleatoria es poco variada, ineficiente o depende de valores semi-estáticos.	0 pts La generación de objetos es estática, está codificada de forma fija (hardcoded) o no funciona adecuadamente.